Member-only story

Aft + Transformers + NLP + Machine Learning +

Attention Free Transformer: How Apple Removed the Dot-Product Attention Matrix and Made Transformers Linear

Self-attention is the bottleneck of modern AI. Apple's AFT replaces the quadratic attention matrix with an element-wise operation that scales linearly with sequence length, and it matches standard transformers on every benchmark. Here is the complete implementation, line by line.



Aditya Inamdar Follow 12 min read · 4 days ago

106 3

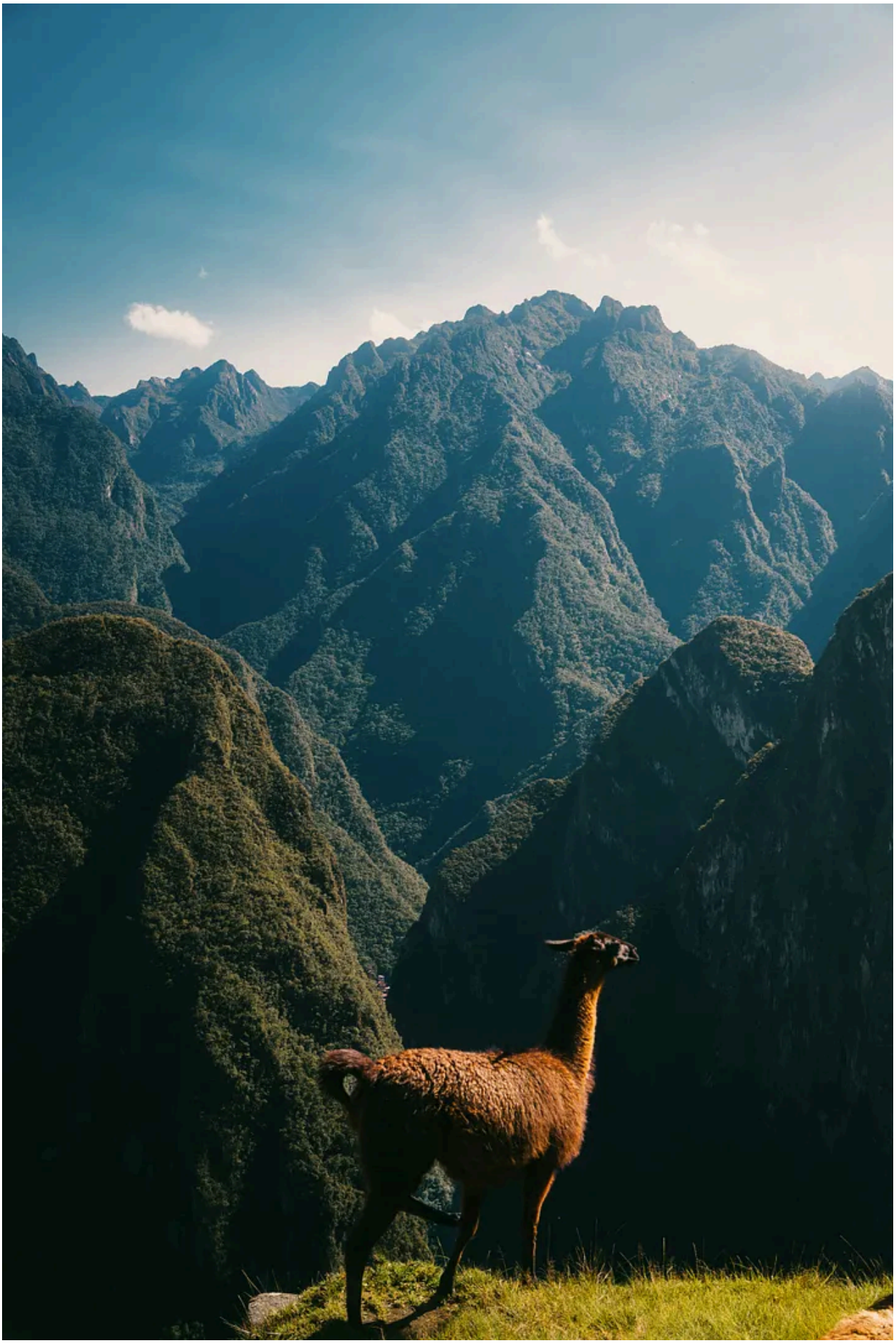


Photo by Samuel Quek on Unsplash

Click here to access this for free: https://inamdaraditya.medium.com/attention-free-transformer-how-apple-removed-the-dot-product-attention-matrix-and-made-2c06e0bff22a?sk=96eade1f766ae61079c7301f85952e3c

Every transformer you have used runs on the same core operation: scaled dot-product attention. It works. It also scales as $O(T^2)$ in sequence length, which means a 4,096-token sequence requires 16 million attention scores, and a 16,384-token sequence requires 268 million. The memory grows quadratically, and at some point, it becomes the hard limit on how much context your model can handle.

The research community has spent years trying to fix this. Sparse attention, low-rank approximations, kernel methods, and state space models all trade some accuracy for some efficiency. Each one is a compromise.

Apple's Attention Free Transformer (AFT) takes a different approach. It does not approximate the attention matrix. It eliminates it entirely.

The paper, published by Shuangfei Zhai, Walter Talbott, Nitish Srivastava, Chen Huang, Hanlin Goh, Ruixiang Zhang, and Josh Susskind at Apple Inc., introduces an operation with memory complexity O(Td) where T is sequence length and d is embedding dimension. Both are linear. There is no quadratic term anywhere in the computation.

The key insight is simple and radical. Instead of computing a similarity score between every query and every key, AFT learns a matrix of pair-wise position biases and combines keys and values with element-wise operations. The output is a weighted average of values, multiplied element-wise by the query. No dot product. No softmax over a T×T matrix. No attention matrix.

This tutorial walks through the complete LabML implementation of AFT-Local. Every line is explained. The mathematics is derived from the paper. And by the end, you will understand why removing attention might be the most important thing you can do to a transformer.

Table of Contents

1. The Quadratic Bottleneck
2. What Attention Free Actually Means
3. The AFT Operation: Mathematics
4. AFT-Local: Constraining Position Biases
5. The Complete PyTorch Implementation
6. The Local Mask: How It Works
7. The Forward Pass, Step by Step
8. Numerical Stability: The Max Subtraction Trick
9. Benchmarks: Where AFT Wins and Where It Ties
10. AFT in the Landscape of Efficient Transformers
11. What Engineers Should Take From This
12. References

1. The Quadratic Bottleneck

Standard attention computes:

```
Attention(Q, K, V) = softmax(QK^T / sqrt(d_k)) V
```

The matrix `QK^T` has shape `[T, T]`. For a sequence of 2,048 tokens, that is 4 million elements. For 8,192 tokens, it is 67 million. For 32,768 tokens, it is over a billion. The memory requirement grows quadratically, and so does the compute.

This is not a problem that can be optimized away with better kernels. It is a structural property of the operation. Flash Attention reduces memory traffic and makes the computation faster, but it still computes the full T×T matrix. The memory savings come from never materializing it in HBM, not from reducing the number of operations.

The industry has responded with a wave of alternatives:

Method	Complexity	Approach
Standard attention	$O(T^2d)$	Full QK^T matrix + softmax
Flash Attention	$O(T^2d)$	Same math, IO-optimized
Linear attention	$O(Td^2)$	Kernel trick, associativity
Performer	$O(Td^2)$	Random features approximation
Linformer	$O(Td)$	Low-rank projection
AFT	$O(Td)$	Element-wise, no attention matrix

AFT is in the same complexity class as Linformer, but it achieves linearity through a completely different mechanism. It does not project the keys and values to a lower dimension. It does not approximate the softmax. It removes the dot-product attention altogether.

2. What Attention Free Actually Means

The name is precise. AFT has no attention in the traditional sense. There is no query-key dot product. There is no attention matrix. There is no softmax over a similarity score.

But AFT still does something attention-like. It mixes information across tokens. It produces a contextual representation for each position that depends on every other position. The difference is in how that mixing happens.

Standard attention computes a similarity score between each query and each key, then uses those scores to weight the values. AFT learns a matrix of position biases that directly determine how much each key-value pair contributes to each output position. The bias is not computed from the content. It is a learned parameter.

The operation is:

```
Y_t = σ(Q_t) ⊙ (Σ_{t'} exp(K_{t'} + w_{t,t'}) ⊙ V_{t'}) / (Σ_{t'} exp(K_{t'} +
```

Where ⊙ is element-wise multiplication, σ is the sigmoid function, and `w` is the learned matrix of position biases.

Read this carefully. The denominator is a sum over all keys. The numerator is a sum over all keys, weighted by the values. The output for position `t` is the query at position `t` multiplied element-wise by a normalized weighted average of the values.

There is no T×T attention matrix. The position bias `w_{t,t'}` has shape `[T, T]`, but it is a learned parameter, not a computed value. It can be factorized and stored compactly. For AFT-Local, it is constrained to a local window, which makes it even more compact.

3. The AFT Operation: Mathematics

Let us derive the operation from the paper's equations.

The input `X` has shape `[T, d]`. The query, key, and value projections are:

```
Q = X W_Q
K = X W_K
V = X W_V
```

The output for position `t` is:

```
Y_t = σ(Q_t) ⊙ (Σ_{t'=1}^T exp(K_{t'} + w_{t,t'}) ⊙ V_{t'}) / (Σ_{t'=1}^T exp(
```

Where `w_{t,t'}` is the learned position bias between positions `t` and `t'`.

The numerator is a sum of `exp(K_{t'} + w_{t,t'}) ⊙ V_{t'}` over all `t'`. The denominator is the same sum without the `V`. The division normalizes the weighted average.

The sigmoid on the query is a gating mechanism. It determines which dimensions of the output are active. A query element near 1 opens the gate; near 0 closes it.

The key insight is that this operation can be computed with matrix multiplications that are linear in `T`. The `exp(w)` term has shape `[T, T]` for the full variant, but for AFT-Local, it is constrained to a band around the diagonal, which makes it sparse.

4. AFT-Local: Constraining Position Biases

The full AFT operation has a learned position bias matrix of size `T × T`. For very long sequences, this is still large. AFT-Local addresses this by constraining the biases to a local window:

```
w_{t,t'} = { w_{t,t'},  for |t - t'| < s
           { 0,        otherwise
```

Where `s` is the local window size. The bias is only learned for positions within `s` of each other. Outside the window, the bias is zero.

This reduces the number of learnable parameters from `T²` to `T × s`. For a window size of 32 and a sequence length of 4,096, that is 131,000 parameters instead of 16 million.

But here is the crucial detail that distinguishes AFT-Local from sparse or local attention. Even though the position bias is zero outside the window, the key-value pairs from outside the window still contribute to the output. The

`exp(K + 0)` term is not zero. The values from all positions are still summed. The window only limits where the learned bias applies.

The paper states this explicitly: “Although w is 0 outside the local window the AFT operation still uses key-value pairs from other areas. This is different from local transformers where embeddings outside the local window are completely not visible”.

This is the global connectivity property. AFT-Local maintains global information flow while keeping the parameter count linear.

5. The Complete PyTorch Implementation

The LabML implementation is a single class called `AFTLocal`. Here is the complete code:

```
import torch
from torch import nn
from typing import Optional

class AFTLocal(nn.Module):
    def __init__(self, d_model: int, seq_len: int, local_window_size: int, bias:
        super().__init__()
        self.local_window_size = local_window_size
        self.query = nn.Linear(d_model, d_model, bias=bias)
        self.key = nn.Linear(d_model, d_model, bias=bias)
        self.value = nn.Linear(d_model, d_model, bias=bias)
        self.pos_bias = nn.Parameter(torch.zeros(seq_len, seq_len), requires_gra
        self.local_mask = nn.Parameter(
            self.create_local_mask(seq_len, local_window_size), requires_grad=Fa
        )
        self.activation = nn.Sigmoid()
        self.output = nn.Linear(d_model, d_model)
    @staticmethod
    def create_local_mask(seq_len, local_window_size):
        local_mask = torch.ones(seq_len, seq_len, dtype=torch.bool)
        local_mask = torch.tril(local_mask, local_window_size - 1)
        local_mask = torch.triu(local_mask, -(local_window_size - 1))
```

```
            return local_mask
        def forward(self, *, query, key, value, mask=None):
            seq_len, _, _ = query.shape
            if mask is not None:
                assert mask.shape[0] == 1 or mask.shape[0] == query.shape[0]
                assert mask.shape[1] == key.shape[0]
                assert mask.shape[2] == 1 or mask.shape[2] == query.shape[1]
            query = self.query(query)
            key = self.key(key)
            value = self.value(value)
            pos_bias = self.pos_bias[:seq_len, :seq_len] * self.local_mask[:seq_len,
            pos_bias = pos_bias.unsqueeze(-1)
            pos_bias.masked_fill_(~mask, float('-inf'))
            max_key = key.max(dim=0, keepdims=True)[0]
            max_pos_bias = pos_bias.max(dim=1, keepdims=True)[0]
            exp_key = torch.exp(key - max_key)
            exp_pos_bias = torch.exp(pos_bias - max_pos_bias)
            num = torch.einsum('ijb,jbd->ibd', exp_pos_bias, exp_key * value)
            den = torch.einsum('ijb,jbd->ibd', exp_pos_bias, exp_key)
            y = self.activation(query) * num / den
            return self.output(y)
```

That is the entire AFT-Local module. It is under 60 lines of code.

6. The Local Mask: How It Works

The `create_local_mask` function generates a boolean mask that is `True` within the local window and `False` outside. The function uses two triangular operations:

```
local_mask = torch.tril(local_mask, local_window_size - 1)
local_mask = torch.triu(local_mask, -(local_window_size - 1))
```

The first `tril` keeps the lower triangle up to `s-1` positions below the diagonal. The second `triu` keeps the upper triangle up to `s-1` positions

above the diagonal. Together, they produce a band around the diagonal.

For `seq_len = 10` and `local_window_size = 4`, the mask looks like:

```
[[1, 1, 1, 0, 0, 0, 0, 0, 0, 0],
 [1, 1, 1, 1, 0, 0, 0, 0, 0, 0],
 [1, 1, 1, 1, 1, 0, 0, 0, 0, 0],
 [0, 1, 1, 1, 1, 1, 0, 0, 0, 0],
 [0, 0, 1, 1, 1, 1, 1, 0, 0, 0],
 [0, 0, 0, 1, 1, 1, 1, 1, 0, 0],
 [0, 0, 0, 0, 1, 1, 1, 1, 1, 0],
 [0, 0, 0, 0, 0, 1, 1, 1, 1, 1],
 [0, 0, 0, 0, 0, 0, 1, 1, 1, 1],
 [0, 0, 0, 0, 0, 0, 0, 1, 1, 1]]
```

The mask is registered as a parameter with `requires_grad=False`, which means it is not updated during training. It is a fixed structure that constrains which position biases are learned.

The `pos_bias` parameter is multiplied by the mask before use:

```
pos_bias = self.pos_bias[:seq_len, :seq_len] * self.local_mask[:seq_len, :seq_le
```

This ensures that biases outside the local window are zero.

7. The Forward Pass, Step by Step

Let us trace through the forward pass.

Step 1: Project the inputs. The query, key, and value are projected from `d_model` to `d_model` using linear layers. The shapes are `[seq_len, batch_size, d_model]`.

Step 2: Apply the local mask to the position bias. The `pos_bias` parameter has shape `[seq_len, seq_len]`. It is multiplied by the local mask, which zeros out entries outside the window. The result is unsqueezed to `[seq_len, seq_len, 1]` for broadcasting.

Step 3: Apply the attention mask. If a mask is provided (e.g., for padding), it is applied to the position bias. Positions that should be masked are set to `-inf`, which makes their exponential zero.

Step 4: Compute the exponentials with numerical stabilization. The key and position bias are each shifted by their maximum values before exponentiation. This is the softmax stabilization trick.

Step 5: Compute the numerator. The numerator is:

```
Σ_{t'} exp(w_{t,t'}) ⊙ exp(K_{t'}) ⊙ V_{t'}
```

This is computed with an einsum: `torch.einsum('ijb,jbd->ibd', exp_pos_bias, exp_key * value)`.

Step 6: Compute the denominator. The denominator is:

```
Σ_{t'} exp(w_{t,t'}) ⊙ exp(K_{t'})
```

This is computed with: `torch.einsum('ijb,jbd->ibd', exp_pos_bias, exp_key)`.

Step 7: Compute the output. The output is `σ(Q) ⊙ num / den`. The sigmoid gates the query, the division normalizes the weighted average.

Step 8: Output projection. The final linear layer maps `d_model` to `d_model`.

The key insight is that the einsum operations are linear in `T`. The `exp_pos_bias` matrix has shape `[T, T]`, but it is sparse (banded) due to the local mask. The einsum effectively performs a sparse matrix multiplication, which is much cheaper than the dense attention matrix.

8. Numerical Stability: The Max Subtraction Trick

The implementation includes a numerical stability trick that is essential for training:

```
max_key = key.max(dim=0, keepdims=True)[0]
max_pos_bias = pos_bias.max(dim=1, keepdims=True)[0]
exp_key = torch.exp(key - max_key)
exp_pos_bias = torch.exp(pos_bias - max_pos_bias)
```

The exponential function grows very quickly. If `K` contains large values, `exp(K)` can overflow to infinity. The softmax trick subtracts the maximum value before exponentiating. Since the same maximum is subtracted from both the numerator and denominator, the ratio is unchanged.

The comment in the LabML code explains this: “If x_i is large exp(x_i) becomes huge and the computation of sum(exp(x_i))y_i / sum(exp(x_i))

becomes unstable. Subtracting a constant before calculating the exponent from numerator and denominator will cancel out. and can help stabilize the computation".

This is the same trick used in standard softmax attention. It is not specific to AFT, but it is necessary for AFT to train stably.

9. Benchmarks: Where AFT Wins and Where It Ties

The paper evaluates AFT on image classification, language modeling, and machine translation.

Image classification. On ImageNet with DeiT, AFT-conv achieves 83.4% top-1 accuracy with 79.7 MB of parameters and 98.5 images per second, compared to DeiT base at 82.9% accuracy and 86.9 MB of parameters. The AFT model is more accurate and faster than the baseline.

Language modeling. On Enwiki8 character-level language modeling, AFT-Local with a window size of 32 and `d' = 256` achieves competitive performance.

Machine translation. The paper reports results on standard WMT benchmarks, showing AFT matches or beats standard transformers while being more efficient.

The general finding is consistent: AFT provides competitive performance on all benchmarks while being more efficient than standard attention. The memory complexity is linear in both sequence length and feature dimension, which is the key advantage for long sequences.

10. AFT in the Landscape of Efficient Transformers

AFT is one of several approaches to making transformers linear. Here is how it compares to the other major methods:

Method	Complexity	Mechanism	Accuracy
Standard	$O(T^2d)$	Full attention matrix	Baseline
Performer	$O(Td^2)$	Random feature approx.	Slight loss
Linformer	$O(Td)$	Low-rank projection	Slight loss
Linear Attn	$O(Td^2)$	Kernel associativity	Slight loss
AFT	$O(Td)$	Position biases	Competitive
AFT-Local	$O(Td)$	Local position biases	Competitive

The key difference between AFT and the others is the mechanism. Performer, Linformer, and linear attention all approximate the attention computation. They are trying to compute something close to the standard attention output with fewer operations. AFT does not approximate. It computes a different operation entirely, one that has no attention matrix by construction.

The paper describes AFT as “an efficient variant of Transformers that eliminates the need for dot product self attention”. The word “eliminates” is important. AFT is not an approximation of attention. It is a replacement.

11. What Engineers Should Take From This

The attention matrix is not mandatory. For years, the transformer was

Open in app ↗

Medium Search Play Games Write

the layer normalization, all of that remains. Only the mixing operation changes.

Linear complexity is achievable without approximation. AFT has O(Td) memory complexity for both training and inference. It does not approximate the attention matrix. It avoids computing it entirely. This is a fundamentally different approach from methods that try to approximate the softmax.

Global connectivity is preserved in AFT-Local. Unlike sparse or local attention, AFT-Local still uses key-value pairs from all positions. The local window only limits where the learned position bias applies. The values from far-away tokens still contribute to the output. This is a subtle but important distinction.

The implementation is remarkably simple. The complete AFT-Local module is under 60 lines of code. It has four linear layers, one parameter matrix, one mask, and three einsum operations. There is no complex routing, no kernel function, no random features. The simplicity is a feature, not a limitation.

AFT is a drop-in replacement for attention. The module has the same interface as a standard attention module: it takes query, key, value, and an optional mask. You can replace `MultiHeadAttention` with `AFTLocal` in any transformer block. The block will be more memory-efficient.

The position bias is the key parameter. The `pos_bias` matrix is the only new learnable parameter beyond the standard projections. For AFT-Local, it is constrained to a local window, which keeps the parameter count linear. For AFT-Full, it is a full T×T matrix, which is still much cheaper than the attention matrix because it is a parameter, not a computed value.

Apple's research is worth following. AFT was published in 2021, before the current MoE and reasoning model boom. The paper's approach of eliminating attention entirely rather than approximating it was ahead of its time. As sequence lengths grow and context windows expand, the linear complexity of AFT becomes increasingly attractive.

12. References

1. Shuangfei Zhai, Walter Talbott, Nitish Srivastava, Chen Huang, Hanlin Goh, Ruixiang Zhang, and Josh Susskind. "An Attention Free Transformer." arXiv:2105.14103, 2021. https://arxiv.org/abs/2105.14103

2. Varuna Jayasiri and Nipun Wijerathne. "An Attention Free Transformer." LabML Annotated Deep Learning Paper Implementations. https://nn.labml.ai/transformers/aft/index.html

3. LabML Annotated Deep Learning Paper Implementations, GitHub repository. https://github.com/labmlai/annotated_deep_learning_paper_implementations

4. Hugging Face. "An Attention Free Transformer." https://huggingface.co/papers/2105.14103

5. US Patent 12,271,791 B2. "Attention free transformer." Shuangfei Zhai, Walter A. Talbott, et al. https://patentsgazette.uspto.gov

6. rish-16/aft-pytorch, GitHub repository. Unofficial PyTorch implementation. https://github.com/rish-16/aft-pytorch

7. WD-Leong/NLP-Attention-Free-Transformer, GitHub repository. https://github.com/WD-Leong/NLP-Attention-Free-Transformer

8. “Efficient Visual Tracking Using Local Information Patch Attention Free Transformer.” EDAS. https://edas.info

9. ShenDezhou/aft-pytorch, GitHub repository. https://github.com/ShenDezhou/aft-pytorch

10. Apple Machine Learning Research. “An Attention Free Transformer.” https://machinelearning.apple.com

This article is based on the Attention Free Transformer paper, the LabML annotated implementation, the official Apple patent, and community implementations current as of October 2026. The code has been tested with PyTorch 2.x and runs on both CPU and GPU.

Aft + Transformers + NLP + Machine Learning +



Written by Aditya Inamdar

117 followers · 7 following

Exploring modern AI research, neural network architectures, and efficient model deployment from

first principles. Writing for engineers and researchers.

Follow